

Photon Backend — One-Page Cheat Sheet · Priyanka R

Stack: Java 17 · Spring Boot 3.2 · MongoDB · AWS Cognito+JWT · Quartz cron · Microservices (portal Java :3000 ↔ crm Node :3001, REST)

Request flow: Filter (JWT/Cognito → SecurityContext → user on request attr) → Controller (@Valid @RequestBody) → Service (business + authZ) → Repository (Spring Data Mongo) → DB → DTO out. Errors → @ControllerAdvice → uniform JSON.

Java (one-liners)

- **Java 17 features:** switch expressions (arrow, returns value), Optional, var, .toList(), text blocks, records, sealed.
- **OOP:** Encapsulation=private fields + isEnabled(); Abstraction=repo interfaces; Inheritance=abstract auth filter; Polymorphism=NotificationService factory (WhatsApp/Email).
- **ArrayList vs LinkedList:** array-backed, O(1) random access = default; LinkedList only for head/mid inserts.
- **HashMap:** buckets by hashCode(), equals() resolves collisions; tree past 8 → O(log n).
- **equals/hashCode:** equal objects → equal hash; override both; matters for map/set keys.
- **Streams:** map/filter/collect/anyMatch; intermediates lazy, run on terminal; anyMatch short-circuits.
- **String immutable:** safe/cacheable/hash key; heavy concat → StringBuilder.
- **Checked vs unchecked:** checked = declare/catch (IOException); unchecked = RuntimeException (I throw APIException).
- **== vs .equals():** reference vs value; always .equals() for objects.

Spring / Spring Boot

- **Spring Boot:** Spring + auto-config + starters + embedded Tomcat; run a main(), no WAR deploy.
- **DI/IoC:** framework creates + wires beans; I use **constructor injection** (private final + @AllArgsConstructor) → immutable, testable.
- **Stereotypes:** @RestController=JSON web, @Service=logic, @Repository=data, @Component=base bean.
- **@Transactional:** method = one DB txn, rollback on runtime ex; proxy-based (self-call bypasses).
- **Validation:** DTO @NotNull/@NotEmpty/@Valid + @Valid @RequestBody → 400 on fail.
- **@ControllerAdvice:** one place, @ExceptionHandler → {status,message,timestamp,path}.
- **Filter/Security:** OncePerRequestFilter validates JWT/Cognito, sets SecurityContext, user on request attr — auth centralized.

MongoDB

- **SQL vs NoSQL:** relational+schema+joins vs flexible JSON docs, scales horizontally; content is document-shaped → Mongo.
- **Query:** MongoRepository derived methods (findByKlassIdAndTargetId); complex → MongoTemplate + Criteria.
- **Index:** avoids full scan; **unique compound** (klass_id, target_id) = one rule per class-target, upsert keys off it.
- **@DBRef:** doc-to-doc link (like FK); can cause N+1 → often store ID + batch-fetch.

WAS / servlet

- **WAS/container:** runtime that receives HTTP, manages servlet lifecycle + threads (Tomcat). Boot **embeds** it in the JAR.
- **WAR vs JAR:** old = WAR into external Tomcat; Boot = executable **JAR** + embedded server (containerizable).
- **Servlet:** Java class handling HTTP in container; Spring's DispatcherServlet routes to controllers.

Microservices

- **vs Monolith:** independently deployable services, own their data, talk over REST/messaging; +independent scaling/isolation, -network/distributed complexity.
- **Comms:** portal↔crm over REST; internal calls use a **service token** header + filter (skip user JWT, still secured).
- **Failures:** timeouts, map to 503 not hang; conceptually retries+backoff, circuit breaker (Resilience4j).
- **Data:** each service owns its DB; share via API/IDs; eventual consistency + events, not one big txn.

AWS / Auth

- **Used:** Cognito (3 user pools), S3 (files), SES/SNS (email/SMS). Signup → create Cognito user → OTP → JWT.
- **JWT:** signed token with claims; issued on login; sent as Bearer header; filter validates sig/expiry; **stateless**.

If stuck (say, don't freeze)

"Haven't used that directly — my understanding is X; closest I've done is [real Robogebra example]; I'd spike it to confirm." · Pull to front-end where you're strong, then bridge back. · "I'm full-stack shipping backend features end-to-end, not a backend specialist — but I own the API, data model, and logic."

HTTP: 200 OK · 201 Created · 400 validation · 401 not-logged-in · 403 forbidden · 404 missing · 409 duplicate · 500 error · 503 downstream-down

Rules: tie every answer to real code · think out loud · never bluff · 11 PM = tired, so **speak slower**.